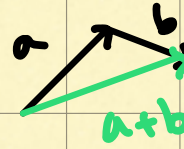


- vector addition = tip to tail
- scalar multip = $s \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} sx \\ sy \end{bmatrix}$
- vector = movement / step in space
- line when you add numbers on a number line
- language to describe space & manipulation of space.



$$\begin{bmatrix} a \\ b \end{bmatrix} + \begin{bmatrix} c \\ d \end{bmatrix} = \begin{bmatrix} a+c \\ b+d \end{bmatrix}$$

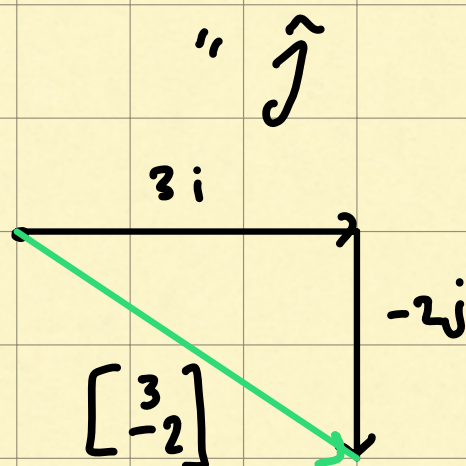
*2: linear combinations, span, & basis vectors

- vector coordinates had this back & forth b/w pairs of numbers & 2D coordinates
- another way to think ab coordinates
- think ab each coordinate as a scalar
- think ab how each one stretches & squishes vectors
- Unit vector in x direction, $\hat{i}$
- " " " " , $\hat{j}$

• x coordinate of a vector is a scalar

stretching $\hat{i}$
- "y"

• ex $\begin{bmatrix} 3 \\ -2 \end{bmatrix} = (3)\hat{i} - 2\hat{j}$



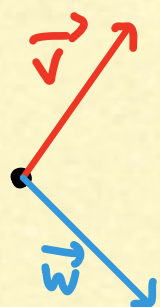
• $\hat{i}$ & $\hat{j}$ are basis vectors of xy coordinates

• when you think of coordinates as scalars...

• basis vectors are what coordinates actually scale!

• what if we chose different basis vectors?

• you could have gotten a new coordinate system



• what are all the vectors you can get by choosing two scalars using each one to scale one of the vectors then adding together

what you get...

- which 2D vectors can be reached by altering choice of scalars?

You can reach every possible 2D vector

- a new pair of basis vectors, still gave a valid way to go back & forth b/w pairs of numbers & 2D vectors
- but association is different than what you get when u use standard basis vectors
- any time we describe vectors numerically, depends on choice of what basis vectors you chose
- linear combination = taking two vectors & scaling them

- The set of all vectors you can reach w/ linear combination of a given pair of vectors, is the span
 - Span of most pairs of 2D vectors, is all 2D vectors, unless
 - vectors are zero (stuck @ origin)
 - vectors face same direction (then span is all vectors in the same direction, the line ...)
 - Span of 2 vectors = all vectors you can reach using only
vector addition
scalar multip
-
- vectors @ points = tail @ origin
 - all 2D vectors all at once = infinite sheet of 2D space

- collection of vectors = pants
- 1 vector = arrow
- Span of 2 3D vectors?
- $\rightarrow$ is it incomplete? do u need 3 vectors?
- their span is all linear combinations of those two vectors
- the span of 2 vectors in 3D space is a 2D plane. (the set of all 3D vectors whose tips sit on that flat sheet)
- the span of 3 vectors in 3D
 - $\rightarrow$ if third vector in span of 1st 2, it's in that plane already, span

does not change. Just like when
you were stuck on a line in 2D when both
vectors lined up same way

→ if 3rd vector outside span of 1st 2,
now you can reach all of 3D space

- you'd use all 3 dimensions to get
all 3D space.

- when you have that redundant vector
it's not adding anything to our span

2D: 1 vector in same direction as other

3D: 1 vector in span of other two

- when you have multiple vectors &
you can remove one w/out reducing
the span -- they are linearly dependent

- 1 vector can be expressed as
linear combination of the other(s).

Since it's already in the span of the others.

- if each vector adds new dimension to span, they are **linearly independent** & cannot be expressed as linear combinations of each other.

- puzzle

basis of a vector space is set of linearly independent vectors that span the full space.

makes sense bc the vectors you can use to reach every vector in the space, are the ones you stretch/squeeze w/ lin comb.
= the basis vectors.

to prove set of vectors is a basis
for a vector space, show that
the set linearly independent & spans the
space

ex: determine if $v_1, v_2, v_3 = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$
form a basis for $\mathbb{R}^3$

→ put into a matrix, compute determinant
or reduce to RREF.

if $\det() \neq 0$, the set is linearly
independent & spans $\mathbb{R}^3$